

ADR-001: Refactorización del módulo de autenticación por vulnerabilidades críticas de seguridad y violaciones a principios Clean Code y SOLID

Contexto

El sistema actual implementa un módulo básico de autenticación que permite registro y login de usuarios utilizando Spring Boot y PostgreSQL. Durante la auditoría de código (Fase 2) y las pruebas funcionales (Fase 3) se identificaron múltiples problemas críticos. Entre ellos: construcción de consultas SQL mediante concatenación de strings (vulnerable a SQL Injection), uso del algoritmo MD5 para hashing de contraseñas, exposición del hash en la respuesta HTTP, ausencia de manejo estructurado de excepciones y validaciones débiles de contraseñas.

Aunque el sistema funciona funcionalmente, las pruebas demostraron que es vulnerable en un entorno productivo. Estas fallas afectan directamente la seguridad de los usuarios, la integridad de la base de datos y la reputación del sistema. Además, se evidencian violaciones a SRP, DIP y buenas prácticas de Clean Code, lo que dificulta la mantenibilidad y escalabilidad futura del módulo.

La situación es urgente porque las vulnerabilidades identificadas son explotables y podrían comprometer datos sensibles en un entorno real, afectando a usuarios finales, al equipo de desarrollo y al negocio por riesgo reputacional y operativo.

Decisión

Se decide refactorizar el módulo de autenticación aplicando principios de seguridad, Clean Code y SOLID.

1. Reemplazar `Statement` por `PreparedStatement`

- Elimina la vulnerabilidad de SQL Injection.
- Reduce el riesgo de manipulación de consultas.
- Mejora seguridad sin reescritura total del módulo.

2. Sustituir MD5 por un algoritmo seguro (BCrypt)

- MD5 es criptográficamente inseguro.
- BCrypt incorpora salt y es resistente a fuerza bruta.
- Alinea el manejo de credenciales con estándares de producción.

3. Eliminar la exposición del hash en la respuesta

- El login retornará solo datos estrictamente necesarios (estado, usuario y/o token).
- Aplica principio de mínima exposición.

4. Implementar validaciones robustas de contraseña

- Longitud mínima ≥ 8 caracteres.
- Reglas básicas de complejidad.
- Disminuye riesgo de credenciales débiles.

5. Separar responsabilidades y mejorar manejo de excepciones

- Implementar manejo global con `@ControllerAdvice`.
- Evitar `throws Exception` genérico y retornar códigos HTTP adecuados.
- Mantener Controller $\rightarrow$ Service $\rightarrow$ Repository con límites claros de responsabilidad.

Consecuencias

Consecuencias positivas

- Eliminación de SQL Injection.
- Protección adecuada de contraseñas.
- Mayor mantenibilidad y claridad del módulo.
- Mejor cumplimiento de buenas prácticas de seguridad.
- Mejor control de errores y respuestas HTTP.

Consecuencias negativas / riesgos

- Tiempo adicional de refactorización y pruebas.
- Posibles regresiones si no se cubre con pruebas funcionales.
- Necesidad de migración/actualización de contraseñas ya almacenadas.
- Ajustes en despliegue y verificación en ambientes.

Alternativas consideradas

1. Reescribir completamente el módulo desde cero

- Descartado por alto costo y tiempo.
- No es necesario si se puede refactorizar incrementalmente.

2. Corregir únicamente SQL Injection

- Descartado porque deja abiertos riesgos graves (MD5, exposición de hash, validaciones débiles).
- Solución parcial no reduce el riesgo global.

3. Mantener el sistema actual y agregar validaciones externas

- Descartado porque la vulnerabilidad estructural seguiría presente en la capa de acceso a datos.

Estado: Propuesto

Fecha: 2026

Autor: Maria Fernanda Robayo Laguna